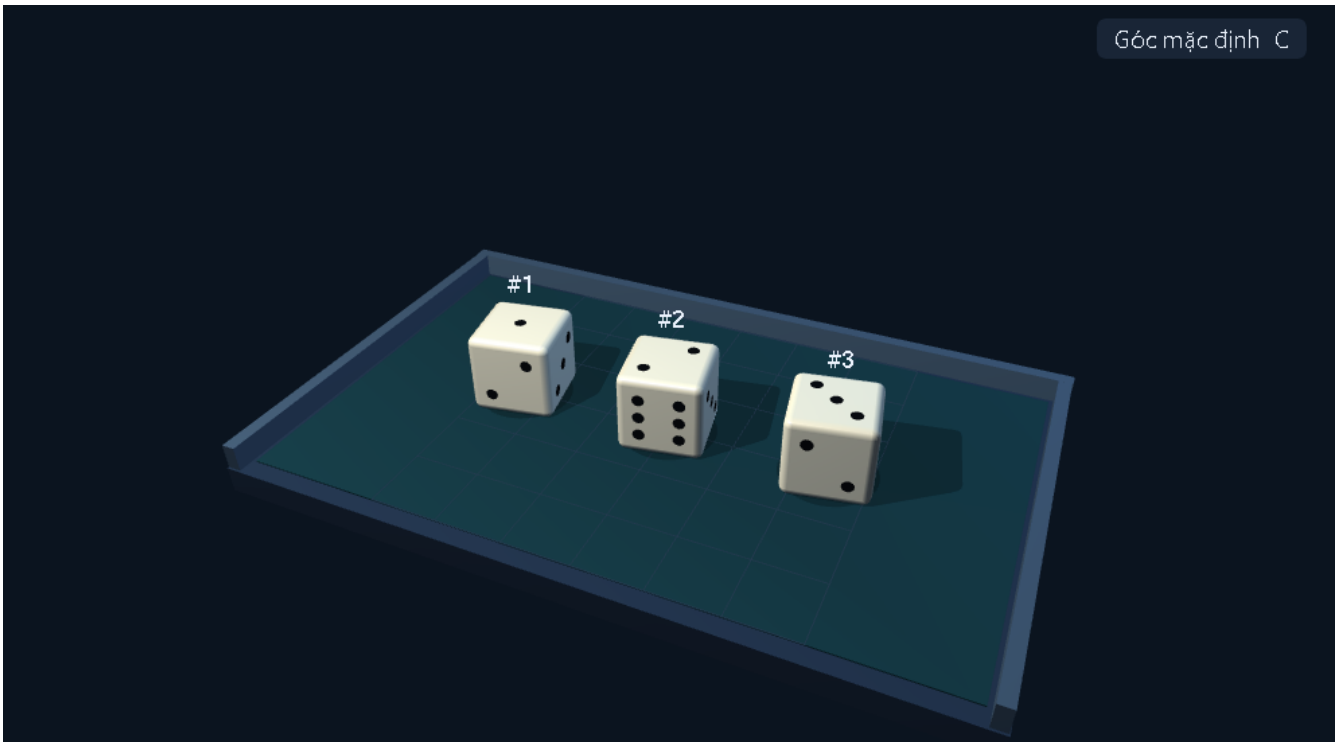
MÔ PHỎNG XÚC XẮC 3D

VÀ TRÒ CHƠI ĐỔ XÚC XẮC

Đồ án môn Đồ họa máy tính

C++17 / OpenGL / FreeGLUT



Phiên bản học thuật 2.0

Mô hình hóa, biến đổi 3D, phép chiếu và hệ quan sát

Tài liệu tham khảo chính: bài giảng 3D Graphics, 51 trang, do người học cung cấp. Các ảnh minh họa là ảnh chụp ứng dụng C++/OpenGL đi cùng báo cáo.

MỤC LỤC NỘI DUNG

1. Giới thiệu và mục tiêu

2. Cơ sở lý thuyết và quy ước

3. Kiến trúc và các đối tượng

4. Mô hình hóa thân xúc xắc

5. Mô hình hóa 6 mặt và các chấm

6. Các phép biến đổi 3D

7. Hoạt ảnh và chạy từng bước

8. Thuật toán mô phỏng đổ xúc xắc

9. Thuật toán trò chơi ghép hướng

10. Quan sát và các phép chiếu

11. Dựng hình, ánh sáng, vật liệu và bóng

12. Lưu trữ dữ liệu

13. Quy trình thực hiện

14. Kiểm thử và đánh giá

15. Kết luận, hạn chế và hướng phát triển

16. Tài liệu tham khảo

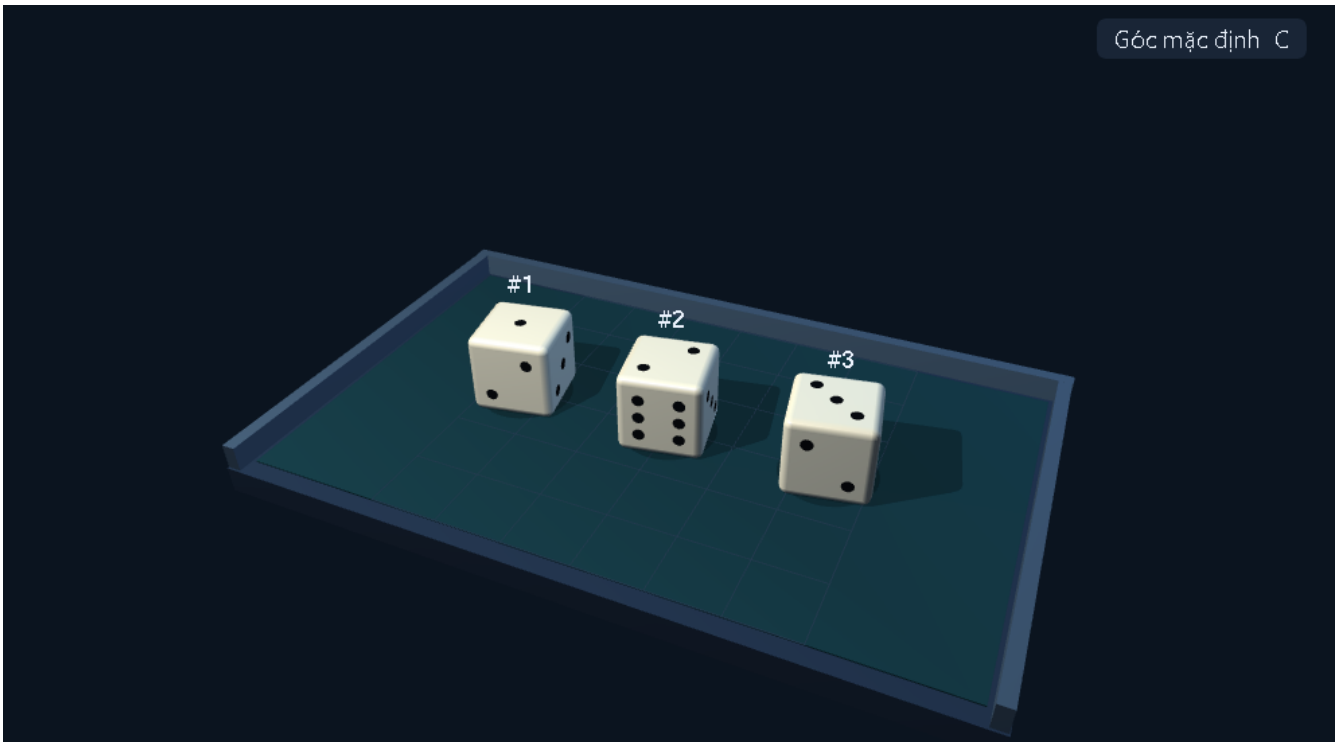
17. Phụ lục ma trận và dữ liệu mẫu

18. Phụ lục thuật toán dạng giả mã

Tài liệu kèm theo gồm slide thuyết trình, kịch bản demo 8-10 phút, câu hỏi bảo vệ và bảng đối chiếu số trang bài giảng.

1. Giới thiệu và mục tiêu

Đề tài xây dựng một môi trường 3D có thể quan sát, thao tác và giải thích bằng các kiến thức đồ họa cơ sở. Xúc xắc là đối tượng phù hợp vì có cấu trúc rõ ràng gồm 6 mặt, có thể mô hình hóa bằng danh sách đỉnh và tam giác, đồng thời cần tịnh tiến và quay khi chuyển động.



Minh họa từ ứng dụng. Ảnh đầy đủ và các chế độ khác nằm trong docs/images.

Sản phẩm có năm trang. Mô phỏng cho phép đổ 1-3 viên lên khay và đọc mặt trên thực tế. Mô hình giải thích thân, mặt và chấm. Biến đổi 3D cho thao tác ma trận có kiểm soát. Phép chiếu theo dõi một đỉnh qua pipeline. Trò ghép hướng yêu cầu quay các viên để khớp mẫu, kết hợp đồ họa với BFS.

Đối tượng sử dụng là sinh viên học đồ họa, giảng viên đánh giá đồ án và người cần quan sát các phép toán 3D. Giao diện tập trung vào cảnh và số liệu của phép đang thực hiện. Thuyết minh chi tiết nằm trong báo cáo để giảm lượng chữ khi thao tác.

Kết quả cần đạt là một ứng dụng chạy trực tiếp, mã nguồn phân chia theo trách nhiệm, dữ liệu có thể xuất và các ví dụ đủ rõ để giải thích lý thuyết. Báo cáo phân biệt những phần mô phỏng gần đúng với những phần được kiểm tra bằng công thức.

2. Cơ sở lý thuyết và quy ước

Tài liệu tham khảo chính là PDF 3D Graphics do người học cung cấp. Bốn nhóm nội dung của tài liệu là biến đổi 3D, mô hình khung dây/mặt đa giác, phép chiếu và hệ quan sát. Bảng đối chiếu chi tiết nằm trong DOI_CHIEU_GIAO_TRINH.md.

Không gian thế giới dùng hệ trục tay phải. +Y hướng lên, khay nằm trên XZ tại y=0. Mặt trước theo logic trò chơi là +Z, không đổi theo camera. Đơn vị độ dài là đơn vị mô phỏng, thời gian tính bằng giây và góc quay đầu vào hiển thị bằng độ.

Điểm thuần nhất là p=(x,y,z,1)^T. Chương trình dùng vector cột và công thức p'=M p. Ma trận 4×4 lưu theo column-major, phần tử hàng r cột c nằm tại m[c*4+r]. Trong PDF, công thức Q=P.M dùng vector hàng. Hai cách tương đương qua chuyển vị và đảo thứ tự tích khi đổi quy ước.

Trong eye space của OpenGL, camera nhìn theo -Z. Ma trận quan sát tự xây bằng lookAt, sau đó nạp vào OpenGL. Bảng theo dõi tọa độ CPU dùng đúng ma trận này, không dùng một công thức camera khác.

3. Kiến trúc và các đối tượng

Chương trình tách phần toán học khỏi mã OpenGL. Vec3 lưu vector ba chiều, Quat lưu hướng quay, Mat4 lưu biến đổi thuần nhất. Vertex gồm position và normal. Mesh gồm vector<Vertex> vertices và vector<unsigned> indices. Dữ liệu này cho phép dùng lại một lưới cho nhiều viên.

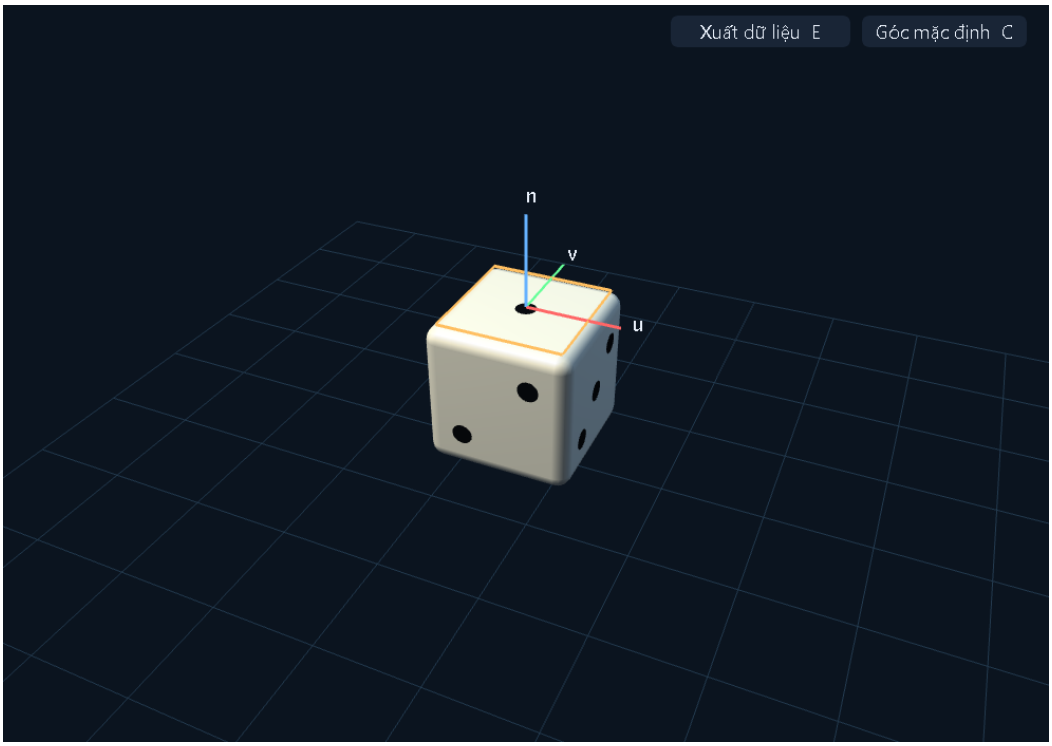
Face lưu value, normal, u và v. Die lưu position, rotation và value. ThrowBody bổ sung vận tốc tuyến tính, vận tốc góc và trạng thái ngủ/ổn định. ThrowSimulation quản lý tối đa ba thân, thời gian và lịch sử. Mỗi viên có ma trận riêng, nhưng dùng chung dữ liệu hình học và lệnh dựng thân.

Lab quản lý danh sách TransformStep, con trỏ chỉnh sửa và con trỏ phát lại. Mỗi bước lưu loại phép, trục, lượng thay đổi và tâm biến đổi. Chọn phép trên giao diện tạo một minh họa đơn, mục Tổ hợp quản lý các ví dụ nhiều phép. FacePuzzle quản lý hướng hiện tại, hướng mục tiêu, số bước và hoạt ảnh. Các đối tượng này không phụ thuộc vào màu nút hay vị trí giao diện.

Luồng một khung hình gồm xử lý đầu vào, cập nhật trạng thái theo thời gian, thiết lập camera, dựng cảnh 3D, dựng giao diện 2D rồi đổi bộ đệm. Hàm timer chỉ đo thời gian và yêu cầu vẽ. Mô phỏng vật lý tiến theo bước cố định thay vì phụ thuộc trực tiếp vào tốc độ vẽ.

4. Mô hình hóa thân xúc xắc

Khối cơ bản có 8 góc hình học, 12 cạnh và 6 mặt vuông. Để từng mặt có pháp tuyến riêng, lưới lưu 4 bản ghi đỉnh cho mỗi mặt, tổng 24 bản ghi. Mỗi hình vuông tách thành hai tam giác, tạo 12 tam giác. Số bản ghi đỉnh vì vậy khác số vị trí góc duy nhất.



Minh họa từ ứng dụng. Ảnh đầy đủ và các chế độ khác nằm trong docs/images.

Thân đẹp hơn dùng khối bo cạnh với bán kích thước h=0,62 và bán kính bo r=0,12. Mỗi mặt chia thành n×n ô, n=20. Từ điểm p trên mặt khối ban đầu, tính c bằng cách chặn từng thành phần của p vào đoạn [-(h-r), h-r]. Sau đó tính normal=normalize(p-c) và vertex=c+r*normal.

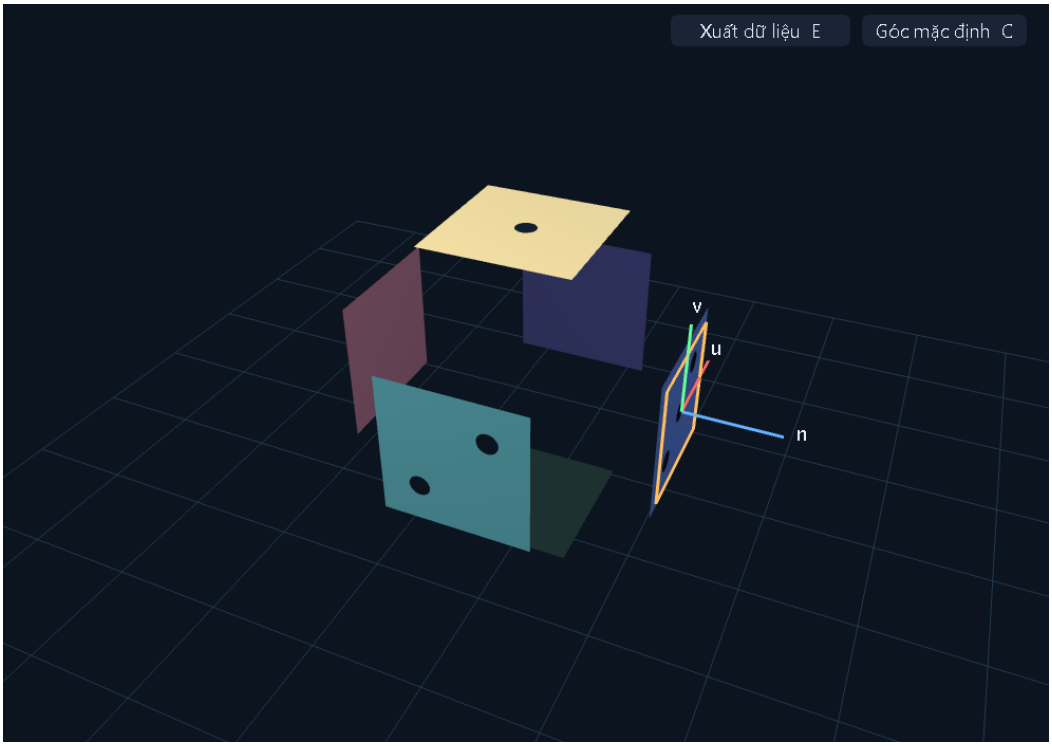
Phép dựng này xem khối bo là hộp bên trong cộng Minkowski với hình cầu bán kính r. Nó tạo phần phẳng ở giữa mặt, cung tròn ở cạnh và phần bo ở góc. Pháp tuyến liên tục trên vùng bo, giúp ánh sáng thay đổi mềm theo bề mặt.

Công thức số lượng là 6(n+1)^2=2646 bản ghi đỉnh và 12n^2=4800 tam giác. Có 14400 chỉ số. Các đường biên giữa mặt có bản ghi trùng vị trí vì sinh từng mặt độc lập. Một Mesh dùng chung cho ba viên nên không tạo lại lưới ở mỗi lượt đổ.

Các tam giác có thứ tự đỉnh hướng ra ngoài. Kiểm tra dùng tích có hướng của hai cạnh chấm với tổng pháp tuyến. Trạng thái wireframe dùng glPolygonMode để xem trực tiếp tam giác, liên hệ với mô hình khung kết nối trong PDF trang 19-26.

5. Mô hình hóa 6 mặt và các chấm

Mỗi mặt có cơ sở trực chuẩn (u,v,n), thỏa u×v=n. Quy ước gán số là 1 ở +Y, 6 ở -Y, 2 ở +Z, 5 ở -Z, 3 ở +X và 4 ở -X. Vì vậy các cặp đối diện luôn có tổng 7.



Minh họa từ ứng dụng. Ảnh đầy đủ và các chế độ khác nằm trong docs/images.

Chấm có tâm trong hệ 2D của mặt. Khoảng cách từ tâm mặt tới cột/hàng chấm là d=0,29. Các số 1,3,5 có chấm giữa. Từ số 2 có hai chấm chéo. Từ số 4 thêm hai góc còn lại. Số 6 có thêm hai chấm ở giữa hai cạnh bên. Tổng số chấm là 1+2+3+4+5+6=21.

Tâm một chấm trong mô hình 3D tính bằng c=n(h+epsilon)+u*a+v*b. Trong mã, epsilon=0,003 để tránh z-fighting. Đường tròn chấm có bán kính 0,088. Điểm trên biên là c+radius*(u*cos(theta)+v*sin(theta)). Dùng GL_TRIANGLE_FAN với 32 tam giác cho mỗi đĩa tròn.

Chấm là hình học thủ tục nằm trên mặt, không cần ảnh texture và không cần sáu mô hình riêng. Chúng không phải lỗ khắc lõm. Riêng 21 chấm thêm 672 tam giác, nên một xúc xắc bo có 5472 tam giác nếu tính cả chấm.

Trang Mô hình cho tách sáu mặt, chọn từng mặt, xem n/u/v và số mặt đối diện. Chế độ tách dùng sáu mặt vuông phẳng nhằm giải thích cấu trúc. Thân bo được quan sát khi thu các mặt lại. Đây là hai biểu diễn phục vụ hai mục đích khác nhau.

6. Các phép biến đổi 3D

Tịnh tiến cộng vector t vào điểm, biểu diễn bằng ba phần tử ở cột cuối của ma trận. Phép tỉ lệ dùng đường chéo sx,sy,sz. Quay dùng ma trận từ quaternion trục-góc, tương ứng quy tắc quay dương tay phải. Đối xứng qua mặt phẳng yz/xy/xz tương ứng đổi dấu x/z/y.

Biến dạng xiên triển khai các quan hệ x'=x+k*y, y'=y+k*z hoặc z'=z+k*x. Với trục XYZ, cả ba quan hệ cùng được áp dụng bằng một ma trận. Đây là một ví dụ affine mở rộng để thấy hình dạng và pháp tuyến thay đổi.

Ma trận quay Rz(alpha) theo vector cột có hai hàng đầu [cos(alpha), -sin(alpha), 0, 0] và [sin(alpha), cos(alpha), 0, 0]. Rx và Ry tính từ cùng quy tắc tay phải. Tỉ lệ dùng S=diag(sx,sy,sz,1). Tịnh tiến T(t) có cột cuối (tx,ty,tz,1)^T.

Quay hoặc tỉ lệ quanh điểm P dùng A=T(P)*B*T(-P). Điểm cần đưa về gốc trước, thực hiện phép B, rồi đưa trở lại. Với quay trục bất kỳ, quaternion dùng hướng k đã chuẩn hóa. Mẫu trong ứng dụng quay 60 độ quanh k=(1,1,0), qua P=(1,0,0). Các điểm nằm trên đường P+lambda*k giữ nguyên sau phép quay.

Tổ hợp phụ thuộc thứ tự. Với vector cột, thực hiện T rồi R có M=R*T. Tâm O đi từ (0,0,0) đến (2,0,0) rồi đến (0,2,0) khi quay +90 độ quanh Z. Nếu thực hiện R rồi T, tâm kết thúc tại (2,0,0). Hai nút mẫu trên trang F4 dựng đúng hai thí nghiệm này.

Mọi phép được áp dụng theo các trục tọa độ cố định của cảnh. Thêm phép A sau trạng thái M bằng M_new=A*M. Chọn một phép mới sẽ chuẩn bị lại trạng thái ban đầu của minh họa đơn. Các chuỗi T rồi R, R rồi T và phép ngược nằm riêng trong mục Tổ hợp để tránh chạy nhầm chuỗi cũ.

Ma trận nghịch đảo tính bằng Gauss-Jordan có chọn phần tử trụ. T^-1=T(-t), R^-1=R(-alpha), S^-1=diag(1/sx,1/sy,1/sz,1). Tỉ lệ bằng 0 làm ma trận suy biến. Ứng dụng không cho tăng/giảm tỉ lệ qua 0 và kiểm tra trường hợp suy biến trong bộ kiểm thử.

Pháp tuyến sau biến đổi không đồng nhất phải dùng n'=normalize((A^-1)^T*n), với A là phần tuyến tính 3×3. Chỉ nhân pháp tuyến bằng A có thể làm nó không còn vuông góc với mặt khi có shear hoặc tỉ lệ không đều. Hàm Mat4::normal tính đúng nghịch đảo chuyển vị.

7. Hoạt ảnh và chạy từng bước

Vị trí là vector, còn hướng quay mô phỏng lưu bằng quaternion đơn vị. Cập nhật hướng bằng q_new=quat(axis,|omega|*dt)*q, rồi chuẩn hóa. Quaternion thuận tiện cho quay theo vận tốc góc, tránh phụ thuộc thứ tự Euler. Khi dựng hình, đổi quaternion thành ma trận và dùng M=T(position)*R(q).

Nội suy quay dùng SLERP theo cung ngắn nhất. Nếu tích vô hướng hai quaternion âm, đổi dấu một quaternion vì q và -q biểu diễn cùng hướng. Khi hai hướng gần nhau, nội suy tuyến tính rồi chuẩn hóa giúp tránh chia cho sin góc quá nhỏ.

Lab dùng con trỏ chỉnh sửa cursor riêng với replayCursor. Khi chạy, model bằng ma trận của các bước đã xong cộng ma trận thành phần của bước đang chạy. Tạm dừng giữ nguyên phase, nên hình không nhảy về mốc trước. Nút Tiến 25% tăng phase thêm 0,25 của phép hiện tại. Sau bốn lần, một phép hoàn tất và chuyển sang phép tiếp theo nếu đang xem tổ hợp. Nút Về đầu đặt phase về 0 mà giữ lựa chọn hiện tại.

Mỗi phép chạy trong 1,6 giây. Nhãn bước, trục được chọn, ma trận hiển thị và đường vết cùng lấy từ activeStep và model hiện tại. Chọn Tịnh tiến, Quay, Tỉ lệ, Đối xứng hoặc Xiên tạo đúng một phép để chạy, không tích lũy ngầm các lựa chọn trước. Đối xứng thực hiện rời rạc ở cuối bước để không đi qua ma trận suy biến. Ví dụ đối xứng đặt hình gốc lệch khỏi mặt phẳng và giữ khung gốc làm mốc để dễ thấy vị trí ảnh.

8. Thuật toán mô phỏng đổ xúc xắc

Người dùng chọn 1-3 viên rồi lấy lực. Lực chuẩn hóa trong khoảng 0,12-1, giữ tối đa 1,35 giây. Lực ảnh hưởng vận tốc ban đầu và vận tốc góc. Chuột phải kéo cung cấp hướng ngang theo góc camera. Bộ sinh mt19937 tạo nhiễu vận tốc ban đầu có seed để tái hiện thí nghiệm.

Vòng lặp dùng dt=1/120 giây, chia bước con không lớn hơn 1/240 giây. Tích phân Euler bán ẩn cập nhật v=v+g*dt trước, rồi p=p+v*dt. Gia tốc trọng trường là (0,-9,81,0). Giảm chấn mũ làm vận tốc giảm dần mà không phụ thuộc quá nhiều vào số bước.

Tiếp xúc mặt bàn và thành dùng khoảng hỗ trợ của khối bo. Với hướng đơn vị d, đưa hướng về hệ local a=R(q)^T*d, rồi h(q,d)=(h-r)*(|ax|+|ay|+|az|)+r. Khi y<h(q,Y), vật xuyên bàn, cần hiệu chỉnh vị trí và áp xung lực. Thành khay kiểm tra tương tự theo X và Z.

Vận tốc tại điểm tiếp xúc là v_contact=v+omega×r_contact. Nếu thành phần theo pháp tuyến đang đi vào bề mặt, tính xung lực với hệ số đàn hồi. Tiếp xúc bàn dùng 0,38, thành dùng 0,62, va rất nhẹ dùng hệ số 0. Ma sát tiếp tuyến bị chặn bởi 0,45 lần xung lực pháp tuyến.

Giữa hai viên, chương trình dùng hình cầu đại diện bán kính 0,78. Kiểm tra khoảng cách tâm nhỏ hơn 1,56, đẩy hai tâm ra khỏi nhau và trao đổi xung lực theo hướng nối tâm. Đây là phương pháp gần đúng để có phản hồi trực quan, có thể tạo khoảng hở ở vài hướng quay. Chưa triển khai SAT/GJK hoặc manifold tiếp xúc chính xác giữa hai khối bo.

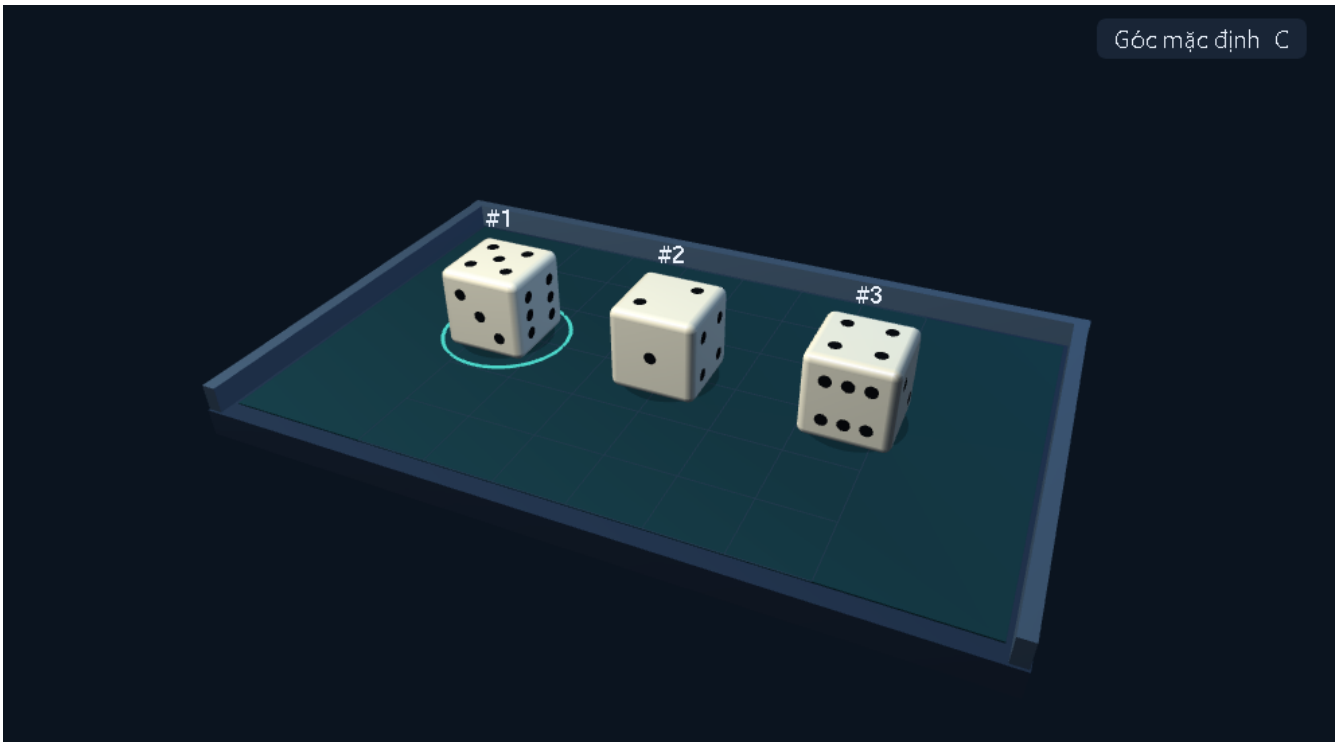
Khi viên ở gần bàn, vận tốc nhỏ và vận tốc góc nhỏ đủ 0,22 giây, chuyển sang ổn định. Chọn mặt đang gần hướng lên nhất và SLERP trong 0,4 giây để mặt đó nằm phẳng. Có ngưỡng thời gian dự phòng để lượt kết thúc. Cần trình bày đây là bước ổn định trực quan có can thiệp, không xem nó là kết quả của bộ giải vật rắn hoàn chỉnh.

Quan trọng nhất, chương trình đọc kết quả từ hướng cuối cùng. Với sáu pháp tuyến mặt local n_i, tính score_i=dot(R(q)*n_i,(0,1,0)). Giá trị của mặt có score lớn nhất là mặt trên. Bản ghi kết quả lấy chính giá trị này, nên số trên giao diện khớp mặt thật của mô hình.

Phần mô phỏng không bảo đảm phân phối các mặt đồng đều. Seed chỉ bảo đảm tái hiện cùng dữ liệu đầu vào trên cùng cách thực thi. Muốn đánh giá công bằng phải thiết kế bộ giải chính xác hơn, lấy nhiều mẫu và thực hiện kiểm định thống kê riêng.

9. Thuật toán trò chơi ghép hướng

Mỗi bài cho ba viên và ba hướng mục tiêu. Một hướng mục tiêu xác định bằng mặt trên +Y và mặt trước +Z. Hai mặt này phải kề nhau. Có 6 lựa chọn cho mặt trên, mỗi lựa chọn có 4 lựa chọn hợp lệ cho mặt trước, tạo 24 hướng rời rạc.



Minh họa từ ứng dụng. Ảnh đầy đủ và các chế độ khác nằm trong docs/images.

Người chơi quay ±90 độ quanh X, Y hoặc Z. Một phép quay là một cạnh của đồ thị. Mỗi đỉnh có tối đa 6 cạnh, chi phí bằng nhau. Khóa trạng thái là topFace*7+frontFace. Khóa chỉ dùng cho các hướng vuông góc rời rạc của trò chơi, không dùng để định danh mọi quaternion trong mô phỏng liên tục.

BFS bắt đầu từ hướng hiện tại, đưa vào hàng đợi, đánh dấu đã thăm. Lấy từng trạng thái ra, thử sáu phép quay, thêm trạng thái chưa gặp và lưu đường đi. Khi gặp mục tiêu, đường đi tìm được có số cạnh ít nhất vì BFS duyệt theo số bước tăng dần.

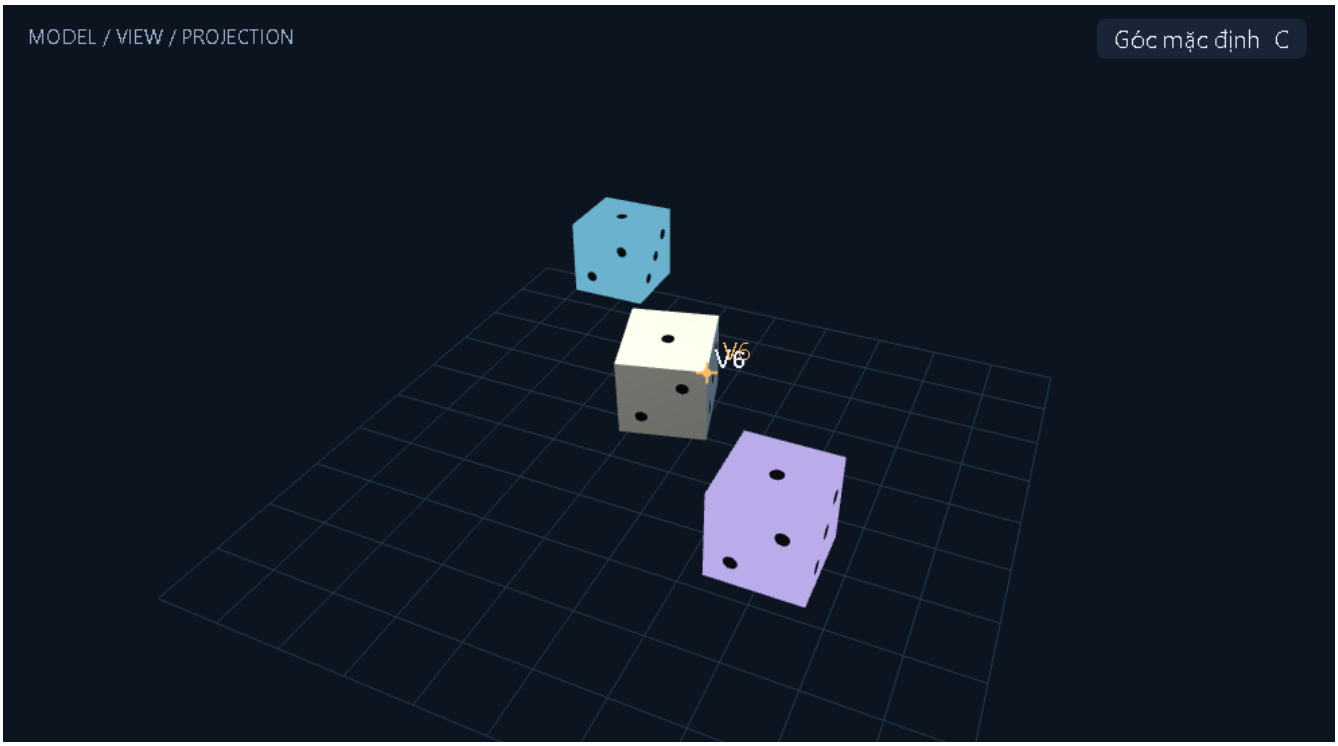
Trên 24 trạng thái, một lần tìm có độ phức tạp O(V+E), với V=24, E không quá 144 cạnh có hướng. Mã hiện tại sao chép đường đi ngắn trong Node để dễ hiểu. Nếu đồ thị lớn hơn có thể lưu parent thay vì cả vector đường đi.

Gợi ý H hiển thị phép quay đầu tiên của đường ngắn nhất. Số bước tối ưu của toàn bài bằng tổng khoảng cách BFS cho ba viên vì từng viên được phép quay độc lập. Hoàn thành đúng tối ưu và không dùng gợi ý đạt 3 sao, trong tối ưu+4 bước đạt 2 sao, còn lại 1 sao. Dùng gợi ý giới hạn tối đa 2 sao.

Hoạt ảnh một phép quay kéo dài 0,24 giây bằng SLERP. Trong lúc đó, khóa lệnh quay tiếp theo để tránh chồng trạng thái. Hướng camera chỉ thay cách quan sát, không thay định nghĩa mặt trên/mặt trước của mục tiêu.

10. Quan sát và các phép chiếu

Camera orbit lưu yaw, pitch và khoảng cách R. Từ các góc, tính vị trí mắt quanh tâm quan sát. Vector forward f=normalize(center-eye), right s=normalize(f×up), up u=s×f. Ma trận view chứa cơ sở s,u,-f và phần tịnh tiến âm tích vô hướng với eye.



Minh họa từ ứng dụng. Ảnh đầy đủ và các chế độ khác nằm trong docs/images.

Projection phối cảnh dùng FOV đứng, aspect, near=0,1 và far=80. Tọa độ clip tính p_clip=P*V*M*p_model. Trong OpenGL, w_clip=-z_eye. Sau đó chia x,y,z cho w để được NDC. Vì vậy các độ lệch x/y nhỏ dần khi độ sâu tăng.

Chiếu trực giao dùng một hộp nhìn. Các đường song song giữ song song, kích thước ảnh không đổi theo độ sâu khi giữ nguyên hộp nhìn. Nút mặt trước/nhìn trên/mặt bên đổi hướng camera và chuyển trực giao để liên hệ hình chiếu kỹ thuật trong PDF trang 31.

Chiếu xiên Cabinet dùng shear trong eye space trước trực giao. Công thức x'=x+0,5*z*cos(45°), y'=y+0,5*z*sin(45°). Thành phần z vẫn giữ để depth test hoạt động. Đây là một trường hợp chiếu song song xiên, không phải phối cảnh một điểm tụ.

Tọa độ pixel trong khung 3D là x=(ndc.x+1)*W/2, y=(1-ndc.y)*H/2. Dấu trừ ở y vì giao diện dùng gốc trên trái. Depth sau ánh xạ là (ndc.z+1)/2. Một đỉnh chỉ được đánh dấu khi nằm trong thể tích nhìn và có w phù hợp.

Trang F5 hiển thị Model, World, Eye, Clip, NDC và Pixel của đỉnh V đang chọn. Dấu vàng là vị trí chiếu trong khung. Đây là dấu tọa độ tham chiếu vẽ đè để giảng giải, nên có thể thấy nó ngay cả khi điểm nằm ở mặt sau đối tượng.

11. Dựng hình, ánh sáng, vật liệu và bóng

Khởi tạo lưới một lần rồi biên dịch vào display list dùng chung. Với mỗi viên, push matrix, nhân ma trận model, gửi thân và chấm rồi pop matrix. Dùng GL_TRIANGLES cho thân, depth buffer để xử lý che khuất, double buffering để giảm nhấp nháy. Tỉ lệ viewport giữ nguyên khi đổi kích thước cửa sổ.

Pipeline cố định OpenGL thực hiện mô hình, quan sát, chiếu, clipping, rasterization, nội suy và depth test. Giao diện 2D dựng sau cảnh với phép chiếu trực giao và depth test tắt. Phần toán học vẫn tự cài, không chỉ ghép các lệnh xoay của thư viện.

Hai đèn gồm đèn chính và đèn bù. Màu mặt phụ thuộc ambient, diffuse theo tích n·l và specular. Vật liệu nhựa có độ bóng trung bình, gỗ ít phản xạ, kim loại có specular và shininess cao. Texture vân gỗ 64×64 sinh bằng hàm sin, không cần ảnh ngoài. Đây là minh họa vật liệu theo mô hình chiếu sáng cố định, chưa phải PBR.

Bóng đổ trên khay dùng ma trận phẳng S=(plane·light)*I-light*plane^T. Hình học được chiếu lên mặt bàn, blend màu đen bán trong suốt. Stencil giới hạn bóng trên khay và giảm việc cộng đen nhiều lần. Bóng tiếp xúc dạng ellipse bổ sung cảm giác sát bàn. Chưa có shadow map hay bóng mềm theo diện tích nguồn sáng.

GL_NORMALIZE giữ pháp tuyến đơn vị sau biến đổi. Trong chế độ học, Mat4::normal dùng nghịch đảo chuyển vị để vẽ các mũi pháp tuyến đúng với shear và tỉ lệ không đều. Khung dây, pháp tuyến và ánh sáng có thể bật/tắt để giải thích tác dụng từng thành phần.

12. Lưu trữ dữ liệu

RAM chứa một Mesh chung, sáu Face, danh sách chấm theo quy tắc, tối đa ba ThrowBody và chuỗi TransformStep. Không lưu riêng hàng nghìn đỉnh thế giới cho mỗi viên. Tọa độ thế giới được tính từ ma trận khi dựng.

Lịch sử mô phỏng tự lưu CSV với các cột id,count,power,d1,d2,d3,total,duration,table_hits,wall_hits. Với ít hơn ba viên, phần dư dùng 0. Khi đọc, kiểm tra số viên, lực, giá trị mặt, tổng, thời gian hữu hạn và id tăng dần. Đọc vào danh sách tạm, chỉ thay dữ liệu hiện tại khi cả file hợp lệ.

Ghi CSV qua file .tmp, sao lưu .bak rồi đổi tên. Nếu đổi tên lỗi, cố phục hồi bản cũ. Khi file có sẵn bị lỗi, ứng dụng giữ nguyên file và ngừng tự ghi đè. Người dùng có thể chọn thư mục mới bằng --data-dir. Đây là cơ chế bảo vệ ở mức ứng dụng, không phải giao dịch cơ sở dữ liệu có bảo đảm chịu mọi tình huống mất điện.

CSV chỉ chứa kết quả và thông số lượt, không chứa tất cả vị trí/quaternion theo thời gian. Khi mở lại, ứng dụng phục hồi số viên và các mặt trên của lượt cuối vào các vị trí chuẩn. Nó không khôi phục chính xác vị trí cuối hay phát lại toàn bộ quỹ đạo cũ.

Nút Xuất dữ liệu E ở F3 xuất dice_body.obj với v, vn và các mặt tam giác f. OBJ chứa thân lưới, chưa chứa chấm thủ tục. Ở F4, OBJ có áp ma trận hiện tại và còn xuất transforms.csv chứa từng bước, tâm biến đổi và trục tùy ý. CSV biến đổi phục vụ đọc/đối chiếu, hiện chưa có giao diện nhập lại.

13. Quy trình thực hiện

Bước đầu xác định quy ước trục, mặt, phép nhân và số liệu chuẩn. Tiếp theo cài toán học Vec3/Quat/Mat4 và kiểm tra với các ví dụ có đáp án. Từ đó sinh lưới, gán pháp tuyến và chấm theo cơ sở từng mặt.

Sau khi dựng được một viên, bổ sung camera và các phép chiếu. Viết phòng biến đổi với lịch sử riêng để tách việc học khỏi vật lý. Sau đó thêm chuyển động, tiếp xúc bàn/thành và hỗ trợ ổn định. Kết quả luôn lấy từ hướng cuối của hình.

Trò chơi dùng lại mô hình và toán quay, xây đồ thị 24 hướng và BFS để có luật rõ ràng. Cuối cùng thiết kế giao diện tiếng Việt, xuất dữ liệu, viết tài liệu và chạy kiểm tra OpenGL thực tế. Các ảnh minh họa trong bộ tài liệu là ảnh của chính ứng dụng.

14. Kiểm thử và đánh giá

Bộ kiểm tra hình học xác nhận cơ sở u×v=n, số chấm đúng, mặt đối tổng 7, hướng tam giác ra ngoài, pháp tuyến đơn vị, số đỉnh/tam giác và hàm support không để lưới xuyên bàn trong các hướng thử.

Kiểm tra ma trận gồm quaternion khớp ma trận, nghịch đảo tổ hợp, pháp tuyến vuông góc tiếp tuyến sau shear, pivot/trục tùy ý cố định và phát lại không làm thay đổi lịch sử. Phép chiếu kiểm tra tâm viewport, quan hệ ảnh theo độ sâu, Cabinet và không chia cho w=0.

BFS kiểm tra 24 hướng duy nhất, mọi cặp hướng tới được và một bài giải tối ưu đạt 3 sao. Mô phỏng kiểm tra nảy, đổi hướng ở thành, va giữa hai viên, khóa thao tác chồng, dừng, tái hiện seed, kết quả khớp mặt trên và CSV bị lỗi không làm hỏng lịch sử trong RAM.

Kiểm tra OpenGL tạo cửa sổ thật, chạy thao tác mô phỏng và các trang, chụp 26 cảnh, kiểm tra glGetError và so sánh ma trận CPU với ma trận đang nạp vào OpenGL. Kết quả chạy cụ thể nằm trong KIEM_THU.md. Những kiểm tra này không thay thế phép đo FPS hoặc kiểm định công bằng thống kê.

15. Kết luận, hạn chế và hướng phát triển

Đồ án thể hiện được một chuỗi hoàn chỉnh từ dữ liệu hình học đến ảnh màn hình và tương tác. Phần học cho phép chứng minh từng phép cơ sở, thứ tự biến đổi, trục bất kỳ, nghịch đảo, camera và phép chiếu. Trò ghép hướng sử dụng BFS có không gian trạng thái và tiêu chí tối ưu rõ ràng.

Giới hạn hiện tại gồm va chạm giữa viên gần đúng bằng cầu, bước ổn định có nội suy, chấm chưa lõm và bóng chỉ trên mặt phẳng. Tài nguyên dựng hình dùng pipeline cố định nên chưa chạy trên core profile hoặc OpenGL ES. Font tiếng Việt tối ưu cho Windows.

Hướng phát triển phù hợp là SAT/GJK và contact manifold, bộ giải xung lực nhiều tiếp xúc, kiểm định thống kê, chuyển sang VAO/VBO/GLSL, shadow mapping, normal mapping cho chấm và lưu replay đầy đủ bằng các snapshot. Những hướng này chưa được tính là tính năng đã triển khai.

16. Tài liệu tham khảo

- Tài liệu môn học do người dùng cung cấp, 3D Graphics, 51 trang, lưu tại reference/CG_3D_Graphics.pdf. Trọng tâm trang 5-16, 19-26 và 28-49.

- FreeGLUT, tài liệu API chính thức, https://github.com/freeglut/freeglut/blob/master/doc/api.md. Tham khảo callback bàn phím, chuột, timer và vòng lặp.

- Microsoft Learn, Font and Text Functions (OpenGL), https://learn.microsoft.com/en-us/windows/win32/opengl/font-and-text-functions. Tham khảo dựng ký tự Unicode bằng font Windows và bitmap display list.

- Mã nguồn của đồ án và kết quả kiểm thử đi cùng bản nộp. Các thông số lưới, mô phỏng và độ phức tạp trong báo cáo lấy từ triển khai này.

17. Phụ lục ma trận và dữ liệu mẫu

Các ma trận sau dùng vector cột. Ký hiệu c=cos(alpha), s=sin(alpha). Phép tịnh tiến và quay quanh Z:

```
       [ 1  0  0  tx ]          [ c -s  0  0 ]
T(t) = [ 0  1  0  ty ]    Rz = [ s  c  0  0 ]
       [ 0  0  1  tz ]          [ 0  0  1  0 ]
       [ 0  0  0   1 ]          [ 0  0  0  1 ]

       [ sx  0  0  0 ]             [ 1  k  0  0 ]
S    = [  0 sy  0  0 ]    Hx(y) = [ 0  1  0  0 ]
       [  0  0 sz  0 ]             [ 0  0  1  0 ]
       [  0  0  0  1 ]             [ 0  0  0  1 ]
```

Ví dụ T(2,0,0) rồi Rz(90 độ), tâm O cho ra (0,2,0). Ma trận M=Rz*T có cột tịnh tiến (0,2,0,1)^T. Phép ngược phải đảo thứ tự tác động.

Một bản ghi Mesh gồm vị trí và pháp tuyến. Các chỉ số xác định tam giác, không chép ba Vertex mới cho mỗi tam giác:

```
struct Vertex { Vec3 position, normal; };
struct Mesh {
    std::vector<Vertex> vertices;
    std::vector<unsigned> indices;
};
// Tam giac i dung vertices[indices[3*i + 0..2]].
```

Một hàng CSV mô phỏng hai viên, mặt 2 và 5, lực 0,5. Số liệu này chỉ minh họa định dạng, không phải phép đo hiệu năng:

```
id,count,power,d1,d2,d3,total,duration,table_hits,wall_hits
1,2,0.5,2,5,0,7,3.2,6,1
```

18. Phụ lục thuật toán dạng giả mã

Mỗi bước mô phỏng cập nhật vận tốc trước vị trí, xử lý tiếp xúc rồi quyết định chuyển sang trạng thái ổn định. Việc đọc mặt trên nằm sau khi hướng cuối đã xác định.

```
while accumulator >= dt:
    for each active die:
        velocity += gravity * dt
        position += velocity * dt
        rotation = normalize(deltaQuaternion * rotation)
        resolveTableAndWallContacts(die)
    resolvePairContactsUsingSphereProxy()
    updateQuietStateAndVisualSettling()
    if all dice sleeping and round not recorded:
        result[i] = topFace(rotation[i])
        appendHistoryOnce(result)
    accumulator -= dt
```

Giả mã này rút gọn phần giảm chấn và chia bước con. Chi tiết thực thi nằm trong ThrowSimulation::integrate và step.

BFS duyệt đồ thị hướng theo từng lớp. Hàng đợi đảm bảo trạng thái có khoảng cách ít bước hơn được xét trước:

```
queue.push(start, emptyPath)
visited.add(key(start))
while queue not empty:
    state, path = queue.popFront()
    if key(state) == key(target): return path
    for turn in {X+90, X-90, Y+90, Y-90, Z+90, Z-90}:
        next = applyTurn(state, turn)
        if key(next) not in visited:
            visited.add(key(next))
            queue.push(next, path + turn)
```

Trong bài này key là cặp mặt trên/mặt trước của hướng rời rạc. Đánh dấu khi thêm vào hàng đợi để tránh đưa cùng một trạng thái vào nhiều lần. Gợi ý lấy path[0] nếu path không rỗng.